Cervical Cancer Prediction

A Complete Machine Learning Pipeline

Dataset: `risk_factors_cervical_cancer.csv`

858 samples × 36 features

Target: `Biopsy` (binary classification)

Pipeline Stages Covered

1. Data Loading & Exploratory Analysis
2. Missing Value Handling
3. Feature Engineering & Scaling
4. Class Imbalance Handling (SMOTE)
5. 12 Model Families Training & Comparison
6. Stratified K-Fold Cross-Validation
7. Hyperparameter Tuning
8. Decision Threshold Optimisation
9. SHAP Interpretability
10. Model Persistence

Google Colab Notebook Documentation

Libraries: scikit-learn · XGBoost · LightGBM · imbalanced-learn · SHAP

Contents

1 Introduction & Problem Statement **3**
1.1 Clinical Background . . . 3
1.2 Key Challenges . . . 3

2 Dataset Description **3**
2.1 Source & Structure . . . 3
2.2 Feature Catalogue . . . 4
2.3 Missing Value Summary . . . 5

3 Environment Setup **5**
3.1 Google Colab Configuration . . . 5
3.2 Library Installation . . . 6
3.3 All Imports . . . 6

4 Data Loading & Exploration **7**

5 Data Preprocessing **7**
5.1 Step 1: Replace "?" with `NaN` . . . 7
5.2 Step 2: Inspect Missing Values . . . 8
5.3 Step 3: Drop High-Missing Columns . . . 8
5.4 Step 4: Median Imputation . . . 8
5.5 Step 5: Define Target Variable . . . 9

6 Train/Test Split, Scaling, and SMOTE **9**
6.1 Stratified Train/Test Split . . . 9
6.2 Feature Scaling with StandardScaler . . . 9
6.3 SMOTE: Synthetic Minority Oversampling . . . 10

7 Model Families **11**
7.1 Overview . . . 11
7.2 Classical Models . . . 11
7.2.1 Logistic Regression . . . 11
7.2.2 Support Vector Machine (SVM) . . . 12
7.2.3 K-Nearest Neighbours (KNN) . . . 12
7.2.4 Gaussian Naive Bayes . . . 12
7.3 Tree-Based Models . . . 13
7.3.1 Decision Tree . . . 13
7.3.2 Random Forest . . . 13
7.3.3 Gradient Boosting . . . 13
7.3.4 XGBoost . . . 14
7.3.5 LightGBM . . . 14
7.4 Neural Network (MLP) . . . 14
7.5 Ensemble Methods . . . 15
7.5.1 Voting Classifier (Soft Voting) . . . 15
7.5.2 Stacking Classifier . . . 15
7.6 Training Loop . . . 16

8 Evaluation Metrics **16**
8.1 Confusion Matrix Terminology . . . 16
8.2 Metric Definitions . . . 17
8.3 Why Accuracy is Misleading Here . . . 17
8.4 PR-AUC vs ROC-AUC . . . 17

9 Visualisations **17**
9.1 ROC Curves . . . 17
9.2 Precision-Recall Curves . . . 18
9.3 Metric Bar Charts . . . 18
9.4 Confusion Matrices . . . 18

10 Stratified K-Fold Cross-Validation **19**
10.1 Why Cross-Validation? . . . 19
10.2 Correct SMOTE + CV Pattern . . . 19

11 Hyperparameter Tuning **19**
11.1 RandomizedSearchCV for XGBoost . . . 19

12 Decision Threshold Tuning **20**
12.1 The Problem with Default Threshold (0.5) . . . 20
12.2 Finding the F1-Optimal Threshold . . . 20
12.3 Clinical Justification . . . 21

13 Feature Importance & SHAP Interpretability **21**
13.1 Feature Importance from Tree Models . . . 21
13.2 SHAP (SHapley Additive exPlanations) . . . 22

14 Model Comparison Summary **22**
14.1 Final Comparison Table . . . 22
14.2 How to Read the Comparison . . . 23

15 Model Saving **23**

16 Pipeline Summary & Best Practices **24**
16.1 Complete Pipeline Diagram . . . 24
16.2 Key Rules Followed . . . 25
16.3 Limitations & Future Work . . . 25

A Appendix: Quick Reference **25**
A.1 Column Index Reference . . . 26
A.2 Saved Model File Reference . . . 27

1. Introduction & Problem Statement

Cervical cancer is the fourth most common cancer in women worldwide, yet it is one of the most preventable through early detection. The objective of this project is to build a robust, reproducible machine learning pipeline that predicts whether a patient's biopsy result is positive (cancer present) based on demographic, behavioural, and clinical risk factors.

1.1 Clinical Background

The **biopsy** is the definitive gold standard for cervical cancer diagnosis. Several screening tests are used in clinical practice:

- **Hinselmann** — Colposcopy-guided visual inspection.
- **Schiller** — Iodine staining test (Schiller's iodine test).
- **Citology** — Pap smear / cytology examination.
- **Biopsy** — Tissue sample examined under microscope; ground truth.

In this pipeline, `Biopsy` is selected as the primary prediction target because it is the most clinically definitive label.

1.2 Key Challenges

1. **Severe class imbalance** — Only ≈6.4% of patients are positive (cancer), giving a 14:1 negative-to-positive ratio.
2. **Missing values encoded as `?`** — Unlike standard `NaN`, the dataset uses the string "?" as a sentinel.
3. **High-dimensional features** — 36 raw features with overlapping/correlated STD indicators.
4. **Medical context** — False negatives (missed cancers) are far more costly than false positives; recall must be prioritised.

2. Dataset Description

2.1 Source & Structure

The dataset originates from the UCI Machine Learning Repository (Hospital Universitario de Caracas, Venezuela). It contains risk factors for 858 patients.

Table 1: Dataset Overview

Property	**Value**
Total rows	858
Total columns	36
Numeric features	34
Target columns	4 (Hinselmann, Schiller, Citology, Biopsy)
Missing value encoding	`?` (string)
Positive rate (Biopsy)	55 / 858 $\approx$ 6.4%
Imbalance ratio	$\approx$ 14:1

2.2 Feature Catalogue

All 36 columns are described below, grouped by category.

Table 2: Complete Feature Descriptions

Column Name	**Type**	**Description**
Age	Integer	Patient age in years
Number of sexual partners	Float	Total number of sexual partners
First sexual intercourse	Float	Age at first sexual intercourse
Num of pregnancies	Float	Number of pregnancies
Smokes	Binary	Whether the patient smokes (0/1)
Smokes (years)	Float	Number of years smoking
Smokes (packs/year)	Float	Smoking intensity in packs per year
Hormonal Contraceptives	Binary	Currently using hormonal contraceptives (0/1)
Hormonal Contraceptives (years)	Float	Years of hormonal contraceptive use
IUD	Binary	Intrauterine device usage (0/1)
IUD (years)	Float	Years of IUD usage
STDs	Binary	Any sexually transmitted disease history (0/1)
STDs (number)	Float	Number of distinct STDs
STDs:condylomatosis	Binary	Condylomatosis diagnosis
STDs:cervical condylomatosis	Binary	Cervical condylomatosis
STDs:vaginal condylomatosis	Binary	Vaginal condylomatosis
STDs:vulvo-perineal condylomatosis	Binary	Vulvo-perineal condylomatosis
STDs:syphilis	Binary	Syphilis diagnosis
STDs:pelvic inflammatory disease	Binary	Pelvic inflammatory disease
STDs:genital herpes	Binary	Genital herpes diagnosis
STDs:molluscum contagiosum	Binary	Molluscum contagiosum
STDs:AIDS	Binary	AIDS diagnosis

(continued)

Column Name	Type	Description
STDs:HIV	Binary	HIV diagnosis
STDs:Hepatitis B	Binary	Hepatitis B
STDs:HPV	Binary	Human Papillomavirus (HPV)
STDs: Number of diagnosis	Integer	Total number of STD diagnoses
STDs: Time since first diagnosis	Float	Years since first STD diagnosis (91% missing)
STDs: Time since last diagnosis	Float	Years since last STD diagnosis (91% missing)
Dx:Cancer	Binary	Prior cancer diagnosis
Dx:CIN	Binary	Cervical intraepithelial neoplasia diagnosis
Dx:HPV	Binary	HPV diagnosis
Dx	Binary	Any composite diagnosis flag
Hinselmann	Binary	Colposcopy result (positive/negative)
Schiller	Binary	Schiller test result
Citology	Binary	Pap smear result
Biopsy	**Binary**	**Biopsy result — primary target**

2.3 Missing Value Summary

Table 3: Missing Values by Column

Column	Missing Count	Missing %
STDs: Time since first diagnosis	787	91.7%
STDs: Time since last diagnosis	787	91.7%
STDs: Number of diagnosis	≈8	≈1%
Number of sexual partners	≈26	≈3%
First sexual intercourse	≈7	≈0.8%
Num of pregnancies	≈56	≈6.5%
Smokes, Smokes (years), Smokes (packs/year)	≈13	≈1.5%
Hormonal Contraceptives (years)	≈13	≈1.5%
IUD, IUD (years)	≈117	≈13.6%
STDs (binary and subtypes)	≈105	≈12.2%

3. Environment Setup

3.1 Google Colab Configuration

The pipeline runs on Google Colab, using Google Drive for persistent storage.

```
from google.colab import drive
```

```
drive.mount('/content/drive')
!mkdir -p /content/drive/MyDrive/cervical-cancer-ml/{data,models}
```

Listing 1: Mount Google Drive and create directory structure

Why this matters: Google Colab sessions are ephemeral — all files are lost when the runtime disconnects. Mounting Drive ensures models and intermediate outputs persist across sessions.

3.2 Library Installation

```
!pip install pandas numpy scikit-learn matplotlib seaborn \
            imbalanced-learn xgboost lightgbm shap joblib -q
```

Listing 2: Install all required packages

Table 4: Key Libraries and Their Roles

Library	Version	Role
pandas	$\geq$1.5	DataFrame manipulation
numpy	$\geq$1.23	Numerical arrays
scikit-learn	$\geq$1.2	Models, preprocessing, evaluation
imbalanced-learn	$\geq$0.10	SMOTE oversampling
xgboost	$\geq$1.7	Gradient-boosted trees
lightgbm	$\geq$3.3	Fast gradient boosting
shap	$\geq$0.41	Model interpretability
matplotlib / seaborn	—	Visualisations
joblib	—	Model serialisation

3.3 All Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings, joblib, shap

warnings.filterwarnings('ignore')

# Preprocessing & validation
from sklearn.model_selection import (
    train_test_split, StratifiedKFold,
    cross_val_score, RandomizedSearchCV
)
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
```

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier,
    VotingClassifier, StackingClassifier
)
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

# Imbalance
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

# Metrics
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix,
    classification_report, average_precision_score,
    precision_recall_curve
)
```

Listing 3: Complete import block

4. Data Loading & Exploration

```python
df = pd.read_csv(
    "/content/drive/MyDrive/cervical-cancer-ml/data/"
    "risk_factors_cervical_cancer.csv"
)
print(f'Shape: {df.shape}')
df.head()
```

Listing 4: Load dataset and display basic information

This cell loads the raw CSV into a pandas `DataFrame`. The initial `df.info()` call reveals that 26 of the 36 columns are of `object` dtype rather than numeric — this is because missing values are encoded as the string `"?"`, which forces pandas to infer the column type as string.

5. Data Preprocessing

5.1 Step 1: Replace "?" with NaN

```python
df = df.replace('?', np.nan)
df = df.apply(pd.to_numeric, errors='coerce')
df = df.astype(float)
```

Listing 5: Replace sentinel values and convert to numeric

> **Critical Detail:** The dataset uses the string `"?"` as a missing-value sentinel, *not* the standard `NaN`. Skipping this step will cause all downstream sklearn operations to throw `ValueError: could not convert string to float`. The three-line block above handles this entirely.

The three operations perform:

1. `replace('?', np.nan)` — Swap every `"?"` string for a proper floating-point NaN.
2. `apply(pd.to_numeric, errors='coerce')` — Coerce any remaining non-numeric strings to NaN.
3. `astype(float)` — Ensure a uniform `float64` dtype across all columns.

5.2 Step 2: Inspect Missing Values

```
missing = df.isnull().sum().sort_values(ascending=False)
missing_pct = (missing / len(df) * 100).round(1)
missing_df = pd.DataFrame({
    'Missing Count': missing, 'Missing %': missing_pct
})
print(missing_df[missing_df['Missing Count'] > 0])
```

Listing 6: Report missing values per column

This generates a sorted report. The two most critical columns are `STDs: Time since first diagnosis` and `STDs: Time since last diagnosis`, each missing in over 91% of rows.

5.3 Step 3: Drop High-Missing Columns

Rationale: Imputing columns that are 91% missing introduces nearly entirely synthetic data, which can mislead the model. The threshold of 40% is a common heuristic; anything above it suggests the column is fundamentally untrustworthy for imputation.

After dropping, the dataset has 34 feature columns plus the 4 target columns.

5.4 Step 4: Median Imputation

```
imputer = SimpleImputer(strategy='median')
df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
print(f'Remaining nulls: {df_imputed.isnull().sum().sum()}')
```

Listing 7: Median imputation for remaining missing values

Why median over mean? Medical data frequently contains outliers (e.g., a patient with 37 pack-years of smoking). The median is robust to such extreme values, whereas the mean would be pulled toward them, creating a biased fill value.

> **Data Leakage Warning:** In this cell, `SimpleImputer.fit_transform` is applied to the full dataset *before* the train/test split. This means the imputed medians are computed from the test set too. In the cross-validation section (Section 10), the correct pattern using `ImbPipeline` is used to avoid this. For a production system,

> always fit the imputer only on training data.

5.5 Step 5: Define Target Variable

```
X = df_imputed.drop(columns=['Biopsy'])
y = df_imputed['Biopsy']

print('Class distribution:')
print(y.value_counts())
print(f'Imbalance ratio: {y.value_counts()[0.0] /
    y.value_counts()[1.0]:.1f}:1')
```

Listing 8: Select Biopsy as the target label

The `Biopsy` column is chosen as the target because:

- It is the definitive clinical test — tissue is examined under a microscope.
- Other targets (Hinselmann, Schiller, Citology) are screening tests with higher false-positive/negative rates.

The resulting class distribution is approximately 803 negatives and 55 positives.

6. Train/Test Split, Scaling, and SMOTE

6.1 Stratified Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Listing 9: Stratified 80/20 split

The `stratify=y` argument is essential. Without it, a random split might place all 55 positive cases in the training set, leaving the test set with no positive examples. Stratification guarantees the 6.4% positive rate is preserved in both folds.

Result:

- Training set: 686 samples (44 positive $\approx$ 6.4%)
- Test set: 172 samples (11 positive $\approx$ 6.4%)

6.2 Feature Scaling with StandardScaler

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

Listing 10: Z-score standardisation

StandardScaler transforms each feature x_j to:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

where μ_j and σ_j are the training-set mean and standard deviation.

Critical rule: `fit_transform` is called only on the training set; `transform` (without fitting) is applied to the test set. Fitting on the test set would leak test-set statistics into the model.

Which models require scaling?

Table 5: Scaling requirement per model family

Model	Needs Scaling?
Logistic Regression	Yes — gradient converges faster
SVM	Yes — kernel computations are distance-based
KNN	Yes — distance metrics are dominated by large-scale featu
MLP (Neural Network)	Yes — gradient descent is scale-sensitive
Naive Bayes	Not required, but harmless
Decision Tree	No — splits are threshold-based, not distance-based
Random Forest	No
Gradient Boosting, XGBoost, LightGBM	No

Since all models share the same scaled input, tree-based models receive slightly unnecessary (but harmless) scaling.

6.3 SMOTE: Synthetic Minority Oversampling

```
smote = SMOTE(random_state=42, k_neighbors=5)
X_train_smote, y_train_smote = smote.fit_resample(
    X_train_scaled, y_train
)
```

Listing 11: Apply SMOTE to the training set only

How SMOTE works: For each minority-class sample x_i, SMOTE:

1. Finds its k nearest minority-class neighbours.
2. Randomly selects one neighbour x_{nn}.
3. Creates a synthetic sample along the line segment: $x_{new} = x_i + \lambda \cdot (x_{nn} - x_i)$, where $\lambda \sim \text{Uniform}(0, 1)$.

Result: After SMOTE, the training set contains ≈1372 samples (686 negative + 686 synthetic positive), perfectly balanced at 50%/50%.

> **Key Rule:** SMOTE is *only* applied to the training set, never to the test set. The test set must reflect the real-world class distribution (14:1) to give honest evaluation metrics.

The `scale_pos_weight` parameter for XGBoost and LightGBM is also computed:

```
scale_pos_weight = (
    float(y_train.value_counts()[0.0]) /
    float(y_train.value_counts()[1.0])
)
# Result: approximately 14.68
```

Listing 12: Compute class imbalance weight for boosting models

This tells the model to penalise misclassifying the minority class ≈14.68× more heavily than the majority class.

7. Model Families

7.1 Overview

Twelve model configurations are trained and evaluated, spanning five families:

Table 6: All Model Families

Family	Model	Key Hyperparameters
Classical	Logistic Regression	`class_weight='balanced'`, `max_iter=1000`
	SVM	`probability=True`, `class_weight='balanced'`
	KNN	`n_neighbors=5`
	Naive Bayes	Default GaussianNB
Tree-based	Decision Tree	`max_depth=5`, `class_weight='balanced'`
	Random Forest	`n_estimators=200`, `class_weight='balanced'`
	Gradient Boosting	`n_estimators=200, lr=0.05, max_depth=4`
	XGBoost	`n_estimators=300, lr=0.05, scale_pos_weight`
	LightGBM	`n_estimators=300, num_leaves=31`
Neural	MLP	`layers=(128,64,32), relu, max_iter=500`
Ensemble	Voting (soft)	RF + XGBoost + LightGBM
	Stacking	RF+XGB+LGBM+LR base; LR meta
Tuned	XGBoost (Tuned)	RandomizedSearchCV, 30 iterations

7.2 Classical Models

7.2.1 Logistic Regression

```
LogisticRegression(
    max_iter=1000,
```

```
    class_weight='balanced',
    random_state=42
)
```

Listing 13: Logistic Regression definition

Logistic Regression models the log-odds of the positive class as a linear combination of features:

$$\log \frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b$$

The sigmoid activation converts this to a probability:

$$P(y=1|\mathbf{x}) = \frac{1}{1+e^{-(\mathbf{w}^\top \mathbf{x}+b)}}$$

`class_weight='balanced'` adjusts sample weights inversely proportional to class frequency, giving 14× more weight to positive examples.

7.2.2 Support Vector Machine (SVM)

```
SVC(probability=True, class_weight='balanced', random_state=42)
```

Listing 14: SVM with RBF kernel

SVM finds the maximum-margin hyperplane separating classes. The default RBF kernel maps features to a higher-dimensional space using:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$$

`probability=True` enables Platt scaling so `predict_proba()` is available for ROC-AUC computation.

7.2.3 K-Nearest Neighbours (KNN)

```
KNeighborsClassifier(n_neighbors=5)
```

Listing 15: KNN with 5 neighbours

KNN classifies a new point by majority vote of its $k = 5$ nearest training neighbours (Euclidean distance). It has no training phase but is computationally expensive at inference. Feature scaling is mandatory because all distances are computed in the original feature space.

7.2.4 Gaussian Naive Bayes

```
GaussianNB()
```

Listing 16: Naive Bayes

Naive Bayes applies Bayes' theorem with the "naive" independence assumption:

$$P(y|\mathbf{x}) \propto P(y) \prod_{j=1}^{d} P(x_j|y)$$

Each $P(x_j|y)$ is modelled as a Gaussian. It is fast, requires no tuning, and serves as a probabilistic baseline.

7.3 Tree-Based Models

7.3.1 Decision Tree

```
DecisionTreeClassifier(
    class_weight='balanced', random_state=42, max_depth=5
)
```

Listing 17: Shallow decision tree

A single CART tree that recursively splits on the feature threshold that maximises information gain (Gini impurity). Capped at `max_depth=5` to prevent overfitting. Highly interpretable but low accuracy on its own.

7.3.2 Random Forest

```
RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)
```

Listing 18: Random Forest with 200 trees

An ensemble of 200 independently grown decision trees. Each tree is trained on a bootstrap sample of the data, and each split considers only $\sqrt{d}$ randomly chosen features. Predictions are averaged (soft voting), which reduces variance dramatically relative to a single tree.

7.3.3 Gradient Boosting

```
GradientBoostingClassifier(
    n_estimators=200, learning_rate=0.05,
    max_depth=4, random_state=42
)
```

Listing 19: sklearn Gradient Boosting

Gradient Boosting builds trees sequentially, each one correcting the residual errors of the previous ensemble:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x})$$

where h_m is a new tree fit to the negative gradient of the loss, and $\eta = 0.05$ is the shrinkage (learning rate).

7.3.4 XGBoost

```
XGBClassifier(
    n_estimators=300, learning_rate=0.05, max_depth=5,
    subsample=0.8, colsample_bytree=0.8,
    scale_pos_weight=scale_pos_weight,
    eval_metric='logloss', random_state=42, verbosity=0
)
```

Listing 20: XGBoost configuration

XGBoost extends Gradient Boosting with regularisation in the objective:

$$\mathcal{L} = \sum_i l(\hat{y}_i, y_i) + \sum_k \left[\gamma T_k + \frac{1}{2}\lambda \|\mathbf{w}_k\|^2 \right]$$

Key additions over vanilla GBM:

- `subsample=0.8` — Row subsampling reduces overfitting.
- `colsample_bytree=0.8` — Column subsampling per tree.
- `scale_pos_weight` — Handles class imbalance natively.
- Second-order (Newton) gradient approximation for faster convergence.

7.3.5 LightGBM

```
LGBMClassifier(
    n_estimators=300, learning_rate=0.05, num_leaves=31,
    scale_pos_weight=scale_pos_weight,
    random_state=42, verbosity=-1
)
```

Listing 21: LightGBM configuration

LightGBM introduces two key algorithmic innovations over XGBoost:

1. **Gradient-based One-Side Sampling (GOSS)** — Keeps all high-gradient instances and random-samples low-gradient ones, reducing computation while preserving accuracy.
2. **Exclusive Feature Bundling (EFB)** — Bundles mutually exclusive sparse features, reducing dimensionality.
3. **Leaf-wise (best-first) tree growth** — Splits the leaf with highest loss reduction, rather than level-wise, producing deeper trees with fewer leaves.

7.4 Neural Network (MLP)

```
MLPClassifier(
    hidden_layer_sizes=(128, 64, 32),
    activation='relu',
    max_iter=500,
    random_state=42
)
```

Listing 22: Multi-Layer Perceptron

A fully connected feedforward network with three hidden layers of sizes 128, 64, and 32. The ReLU activation function $f(z) = \max(0, z)$ avoids the vanishing gradient problem. The output is a single sigmoid neuron for binary classification.

Trained by backpropagation using stochastic gradient descent (Adam optimiser by default).

7.5 Ensemble Methods

7.5.1 Voting Classifier (Soft Voting)

```
VotingClassifier(
    estimators=[
        ('rf',   RandomForestClassifier(...)),
        ('xgb',  XGBClassifier(...)),
        ('lgbm', LGBMClassifier(...)),
    ],
    voting='soft'
)
```

Listing 23: Soft voting ensemble

The Voting Classifier averages the predicted class probabilities across all three constituent models:

$$P(y = 1|\mathbf{x}) = \frac{1}{3}\left[P_{RF}(y = 1|\mathbf{x}) + P_{XGB}(y = 1|\mathbf{x}) + P_{LGBM}(y = 1|\mathbf{x})\right]$$

Why soft voting? Soft voting utilises the confidence of each model, whereas hard voting only uses the binary predicted label. Soft voting consistently outperforms hard voting when the base models are well-calibrated.

7.5.2 Stacking Classifier

```
StackingClassifier(
    estimators=[
        ('rf',   RandomForestClassifier(...)),
        ('xgb',  XGBClassifier(...)),
        ('lgbm', LGBMClassifier(...)),
        ('lr',   LogisticRegression(...)),
    ],
    final_estimator=LogisticRegression(
        max_iter=1000, class_weight='balanced', random_state=42
    ),
    cv=5
)
```

Listing 24: Stacking with Logistic Regression meta-learner

Stacking is a two-level ensemble:

1. **Level 0 (Base learners):** Four diverse models generate out-of-fold predictions using 5-fold cross-validation.
2. **Level 1 (Meta-learner):** A Logistic Regression is trained on the stacked predictions from Level 0.

The `cv=5` parameter ensures base learner predictions for the meta-learner are generated out-of-fold, preventing overfitting.

7.6 Training Loop

```
results = {}

for name, model in models.items():
    model.fit(X_train_smote, y_train_smote)
    y_pred  = model.predict(X_test_scaled)
    y_proba = model.predict_proba(X_test_scaled)[:, 1]
    results[name] = {
        'model':     model,
        'y_pred':    y_pred,
        'y_proba':   y_proba,
        'accuracy':  accuracy_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred, zero_division=0),
        'recall':    recall_score(y_test, y_pred, zero_division=0),
        'f1':        f1_score(y_test, y_pred, zero_division=0),
        'roc_auc':   roc_auc_score(y_test, y_proba),
        'pr_auc':    average_precision_score(y_test, y_proba),
    }
```

Listing 25: Unified training and evaluation loop

Every model is evaluated on the *original* (unbalanced) test set using both threshold-based metrics (`accuracy`, `precision`, `recall`, `f1`) and threshold-free metrics (`roc_auc`, `pr_auc`).

8. Evaluation Metrics

8.1 Confusion Matrix Terminology

For binary classification with positive = cancer:

$$\text{Confusion Matrix} = \begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}$$

- **TP** (True Positive): Cancer correctly identified.
- **TN** (True Negative): Healthy correctly identified.
- **FP** (False Positive): Healthy wrongly flagged as cancer (false alarm).
- **FN** (False Negative): Cancer missed — the most dangerous error in clinical screening.

8.2 Metric Definitions

Table 7: Evaluation Metrics Used

Metric	Formula	Interpretation
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall correctness (misleading on imbalanced data)
Precision	$\frac{TP}{TP+FP}$	Of predicted positives, how many are real
Recall (Sensitivity)	$\frac{TP}{TP+FN}$	Of all true positives, how many are caught
F1 Score	$\frac{2 \cdot P \cdot R}{P+R}$	Harmonic mean of Precision and Recall
ROC-AUC	$\int_0^1 TPR\, d(FPR)$	Threshold-free discriminative ability
PR-AUC	$\int_0^1 P\, dR$	Precision-Recall tradeoff; best for imbalanced data

8.3 Why Accuracy is Misleading Here

A model that predicts "no cancer" for every patient achieves:

$$\text{Accuracy} = \frac{803}{858} \approx 93.6\%$$

yet has zero clinical utility (Recall = 0). This is why the primary metric is **ROC-AUC** and secondary metrics are **Recall** and **F1 Score**.

8.4 PR-AUC vs ROC-AUC

For highly imbalanced datasets, PR-AUC is more informative than ROC-AUC because:

- ROC-AUC can be inflated by the large number of true negatives.
- PR-AUC focuses exclusively on the performance on positive examples.
- A baseline model in PR space has PR-AUC $\approx$ positive rate (≈ 0.064), making any model trivially better than random.

9. Visualisations

9.1 ROC Curves

```
fig, ax = plt.subplots(figsize=(11, 7))

for name, r in results.items():
    fpr, tpr, _ = roc_curve(y_test, r['y_proba'])
    ax.plot(fpr, tpr, label=f"{name} (AUC={r['roc_auc']:.3f})")

ax.plot([0, 1], [0, 1], 'k--', label='Random')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curves     All Models')
ax.legend(loc='lower right', fontsize=8)
plt.tight_layout()
plt.show()
```

Listing 26: Plot ROC curves for all models

The ROC curve plots TPR (Recall) against FPR at all possible classification thresholds. A perfect classifier has AUC = 1.0; a random classifier produces AUC = 0.5 (the diagonal dashed line). Tree-based ensemble models (Random Forest, XGBoost, LightGBM) typically cluster near the top-left corner, indicating high discriminative ability.

9.2 Precision-Recall Curves

```
for name, r in results.items():
    precision, recall, _ = precision_recall_curve(y_test, r['y_proba'])
    ax.plot(recall, precision, label=f"{name}
        (PR-AUC={r['pr_auc']:.3f})")

baseline = y_test.mean()  # ~0.064
ax.axhline(y=baseline, color='k', linestyle='--',
           label=f'Baseline ({baseline:.3f})')
```

Listing 27: Precision-Recall curves

The dashed horizontal baseline at ≈ 0.064 represents the PR-AUC of a random classifier. Any curve substantially above this demonstrates genuine learning. The steeper the curve (high precision at high recall), the better the model.

9.3 Metric Bar Charts

```
metrics_to_plot = ['ROC-AUC', 'F1 Score', 'Recall', 'Precision']
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
```

Listing 28: 2x2 grid of metric comparison bar charts

Four horizontal bar charts arranged in a 2×2 grid allow simultaneous visual comparison of all models across the four most important metrics. Each bar is annotated with its exact numeric value.

9.4 Confusion Matrices

```
ncols = 4
nrows = (n + ncols - 1) // ncols
fig, axes = plt.subplots(nrows, ncols, figsize=(ncols*4, nrows*3.5))

for i, (name, r) in enumerate(results.items()):
    cm = confusion_matrix(y_test, r['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i],
                xticklabels=['Neg', 'Pos'], yticklabels=['Neg', 'Pos'])
```

Listing 29: Grid of confusion matrices for all models

All confusion matrices are plotted in a single figure. Each cell shows the integer count. Pay particular attention to the bottom-left cell (FN — missed cancers): a clinically safe model should minimise this value even at the cost of more FP.

10. Stratified K-Fold Cross-Validation

10.1 Why Cross-Validation?

A single train/test split can be misleading if the random split happens to favour or disfavour a particular model. K-fold cross-validation evaluates each model k times on different data splits, giving a mean and standard deviation of performance.

10.2 Correct SMOTE + CV Pattern

```
from imblearn.pipeline import Pipeline as ImbPipeline

pipe = ImbPipeline([
    ('smote', SMOTE(random_state=42, k_neighbors=5)),
    ('clf',   model)
])

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
auc_scores = cross_val_score(
    pipe, X_scaled_full, y, cv=skf,
    scoring='roc_auc', n_jobs=-1
)
f1_scores = cross_val_score(
    pipe, X_scaled_full, y, cv=skf,
    scoring='f1', n_jobs=-1
)
```

Listing 30: SMOTE inside ImbPipeline prevents data leakage

> **Why `ImbPipeline` is essential:** If SMOTE is applied to the full dataset before splitting, synthetic samples could leak into the validation fold. Using `ImbPipeline`, SMOTE is applied exclusively to the training fold inside each CV iteration. This is the only correct way to combine SMOTE with cross-validation.

Result format:

$$\text{e.g., XGBoost: AUC} = 0.9521 \pm 0.0312$$

The mean tells us the expected performance; the standard deviation tells us stability across folds.

11. Hyperparameter Tuning

11.1 RandomizedSearchCV for XGBoost

```
param_dist = {
    'n_estimators':      [100, 200, 300, 500],
    'max_depth':         [3, 4, 5, 6],
    'learning_rate':     [0.01, 0.05, 0.1, 0.2],
    'subsample':         [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'min_child_weight': [1, 3, 5],
    'gamma':             [0, 0.1, 0.3],
```

```
}

random_search = RandomizedSearchCV(
    xgb_base,
    param_distributions=param_dist,
    n_iter=30,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring='roc_auc',
    random_state=42,
    n_jobs=-1,
    verbose=1
)

random_search.fit(X_train_smote, y_train_smote)
```

Listing 31: Hyperparameter search space and RandomizedSearchCV

RandomizedSearchCV vs GridSearchCV:

Method	Search Space	Trials
GridSearchCV	All combinations ($4 \times 4 \times 4 \times 3 \times 3 \times 3 \times 3 = 5184$)	$5184 \times 5 = 25920$ fits
RandomizedSearchCV	Random sample (`n_iter=30`)	$30 \times 5 = 150$ fits

RandomizedSearchCV samples 30 random combinations from the search space, fitting each with 5-fold CV. This is 173× faster than a full grid search while typically finding a near-optimal configuration.

Hyperparameter meanings:

- `n_estimators` — Number of trees; more trees = more accuracy but slower.
- `max_depth` — Maximum tree depth; deeper = more complex, prone to overfitting.
- `learning_rate` (η) — Shrinkage applied to each tree's contribution.
- `subsample` — Fraction of training rows sampled per tree.
- `colsample_bytree` — Fraction of features sampled per tree.
- `min_child_weight` — Minimum sum of instance weight in a leaf; higher = more conservative.
- `gamma` — Minimum loss reduction required for a split; higher = more conservative.

12. Decision Threshold Tuning

12.1 The Problem with Default Threshold (0.5)

By default, a sample is predicted positive if $P(y = 1|\mathbf{x}) \geq 0.5$. For a dataset where positives represent only 6.4% of cases, this threshold is suboptimal because the model is calibrated on SMOTE-balanced data (50/50) but deployed on real data (6.4% positive).

12.2 Finding the F1-Optimal Threshold

```
precisions, recalls, thresholds = precision_recall_curve(
    y_test, best_proba
)

# F1-optimal
f1_scores_thr = 2 * (precisions[:-1] * recalls[:-1]) / \
                (precisions[:-1] + recalls[:-1] + 1e-9)
best_thr = thresholds[f1_scores_thr.argmax()]

# High-recall threshold (recall >= 0.90)
recall_idx      = np.where(recalls[:-1] >= 0.90)[0]
high_recall_thr = thresholds[recall_idx[-1]] if len(recall_idx) else
    best_thr
```

Listing 32: Precision-recall threshold analysis

Effect of threshold adjustment:

- **Lower threshold** → More positive predictions → Higher Recall, Lower Precision.
- **Higher threshold** → Fewer positive predictions → Lower Recall, Higher Precision.

12.3 Clinical Justification

In cancer screening, the cost asymmetry is extreme:

- **FN (missed cancer)**: Patient goes untreated → disease progresses → potentially fatal.
- **FP (false alarm)**: Patient undergoes unnecessary follow-up tests → anxiety, cost.

Therefore, the **high-recall threshold** ($\geq 90\%$ recall) is preferred for clinical deployment, even though it increases false positives. The final clinical threshold should be set in consultation with domain experts.

13. Feature Importance & SHAP Interpretability

13.1 Feature Importance from Tree Models

```
for ax, model_name in zip(axes, ['Random Forest', 'XGBoost']):
    model_obj = results[model_name]['model']
    importances = model_obj.feature_importances_
    fi_df = pd.DataFrame({'Feature': feature_names, 'Importance':
        importances})
    fi_df = fi_df.sort_values('Importance', ascending=False).head(15)
    ax.barh(fi_df['Feature'][::-1], fi_df['Importance'][::-1])
```

Listing 33: Extract and plot feature importances

Feature importance for tree models is computed as the total reduction in the splitting criterion (Gini impurity for Random Forest; log-loss for XGBoost) attributable to each feature, averaged across all trees and normalised to sum to 1.

Typical top features for this dataset include:

- `Schiller`, `Hinselmann`, `Citology` — Other diagnostic tests (correlated with Biopsy).
- `Dx:Cancer`, `Dx:CIN`, `Dx:HPV` — Prior diagnoses.
- `Age`, `Number of sexual partners` — Demographic risk factors.
- `STDs:syphilis`, `STDs:HIV` — High-risk STD indicators.

13.2 SHAP (SHapley Additive exPlanations)

```
explainer   = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_test_scaled[:200])

shap.summary_plot(shap_values, X_test_scaled[:200],
                  feature_names=feature_names, plot_type='bar')
shap.summary_plot(shap_values, X_test_scaled[:200],
                  feature_names=feature_names)  # beeswarm
```

Listing 34: SHAP analysis for XGBoost

SHAP values are grounded in cooperative game theory. For a prediction $f(\mathbf{x})$, the SHAP value ϕ_j for feature j is:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[f_{S \cup \{j\}}(\mathbf{x}) - f_S(\mathbf{x}) \right]$$

This computes each feature's marginal contribution across all possible subsets, guaranteeing:

- **Local accuracy**: $\sum_j \phi_j + \phi_0 = f(\mathbf{x})$ (SHAP values sum to the model output).
- **Consistency**: If a feature's impact increases in a new model, its SHAP value does not decrease.
- **Missingness**: Features absent from the model get $\phi_j = 0$.

Two SHAP plots are generated:

1. **Bar plot** — Shows mean $|\phi_j|$ per feature, giving global importance ranking.
2. **Beeswarm plot** — Each point is one patient, coloured by feature value (red = high, blue = low). Points to the right mean the feature pushed the prediction *toward* cancer; points to the left pushed it *away*.

14. Model Comparison Summary

14.1 Final Comparison Table

```
final_df = pd.DataFrame({
    name: {
        'Accuracy':  round(r['accuracy'],  4),
        'Precision': round(r['precision'], 4),
        'Recall':    round(r['recall'],    4),
        'F1 Score':  round(r['f1'],        4),
        'ROC-AUC':   round(r['roc_auc'],   4),
        'PR-AUC':    round(r['pr_auc'],    4),
    }
    for name, r in results.items()
```

```
}).T.sort_values('ROC-AUC', ascending=False)
```

Listing 35: Final model comparison

Expected ordering (typical):

1. XGBoost (Tuned) / Stacking Ensemble — highest AUC
2. Voting Ensemble, LightGBM, XGBoost — strong tree ensembles
3. Random Forest, Gradient Boosting — solid baselines
4. MLP, SVM — moderate
5. Logistic Regression, KNN — fast baselines
6. Naive Bayes, Decision Tree — weakest on this task

14.2 How to Read the Comparison

- **ROC-AUC > 0.95** — Excellent discriminative power.
- **Recall > 0.80** — Catching 80%+ of true cancer cases.
- **PR-AUC > 0.60** — Strong performance on the minority class.
- High accuracy (e.g., 96%) with low recall (e.g., 30%) = model mostly predicts negatives.

15. Model Saving

```
save_dir = '/content/drive/MyDrive/cervical-cancer-ml/models/'

# Save scaler (always needed at inference time)
joblib.dump(scaler, save_dir + 'scaler.pkl')

# Save each model
model_files = {
    'Random Forest':      'rf_model.pkl',
    'XGBoost':            'xgb_model.pkl',
    'XGBoost (Tuned)':    'xgb_tuned_model.pkl',
    'Stacking Ensemble': 'stacking_model.pkl',
    # ... all others ...
}

for name, filename in model_files.items():
    if name in results:
        joblib.dump(results[name]['model'], save_dir + filename)
```

Listing 36: Save all models to Google Drive

Loading a saved model for inference:

```
import joblib
import numpy as np

# Load artefacts
scaler    = joblib.load('/path/to/scaler.pkl')
model     = joblib.load('/path/to/xgb_tuned_model.pkl')

# Prepare new patient data (34 features, same order as training)
```

```python
new_patient = np.array([[28, 2, 17, 1, 0, 0, 0, 1, 2, 0,
                         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                         0, 0, 0, 0]])

# Scale and predict
new_scaled = scaler.transform(new_patient)
prob_cancer = model.predict_proba(new_scaled)[0, 1]
print(f'Cancer probability: {prob_cancer:.4f}')
```

Listing 37: Load and use a saved model

16. Pipeline Summary & Best Practices

16.1 Complete Pipeline Diagram

Raw CSV (858 rows, 36 cols, "?" for missing)

↓

Replace "?" → NaN, to_numeric()

↓

Drop 2 high-missing cols (>40%) → 34 feature cols

↓

Median Imputation (SimpleImputer)

↓

Target separation: X (34 features), y (Biopsy)

↓

Stratified 80/20 split (random_state=42)

↓

StandardScaler (fit on train, transform both)

↓

SMOTE (train set only) → 686 neg + 686 pos

↓

Train 10 models + 2 ensembles + 1 tuned

↓

Evaluate on original test set (172 samples, 6.4% positive)

↓

Cross-validation (5-fold, SMOTE inside ImbPipeline)

↓

Threshold tuning + SHAP

↓

Save best model + scaler to Drive

16.2 Key Rules Followed

1. **No data leakage** — Scaler fitted only on training data; SMOTE inside ImbPipeline during CV.
2. **Stratification** — Class ratio preserved in all splits.
3. **Correct metrics** — ROC-AUC and PR-AUC used instead of accuracy.
4. **SMOTE on train only** — Test set always reflects real-world distribution.
5. **Multiple models** — Diversity reduces the risk of selecting a model that overfits to a particular split.
6. **Threshold tuning** — Clinical deployment requires adapting the decision boundary beyond 0.5.
7. **Interpretability** — SHAP provides actionable insights for clinicians.

16.3 Limitations & Future Work

- **Small dataset** — 858 samples is small for neural networks; more data would improve MLP.
- **Feature selection** — RFE or SHAP-based feature pruning could remove noisy STD sub-features.
- **Calibration** — Predicted probabilities should be calibrated (Platt scaling or isotonic regression) for clinical use.
- **Prospective validation** — Performance on a held-out external cohort is needed before clinical deployment.
- **Multi-target learning** — Predicting all four targets (Hinselmann, Schiller, Citology, Biopsy) jointly using multi-label classification could improve performance.
- **Optuna** — Bayesian hyperparameter optimisation (Optuna) would outperform RandomizedSearchCV with fewer trials.

A. Appendix: Quick Reference

A.1 Column Index Reference

Table 8: Feature column indices (0-indexed, after dropping 2 high-missing cols)

Index	Column	Type
0	Age	Continuous
1	Number of sexual partners	Continuous
2	First sexual intercourse	Continuous
3	Num of pregnancies	Continuous
4	Smokes	Binary
5	Smokes (years)	Continuous
6	Smokes (packs/year)	Continuous
7	Hormonal Contraceptives	Binary
8	Hormonal Contraceptives (years)	Continuous
9	IUD	Binary
10	IUD (years)	Continuous
11	STDs	Binary
12	STDs (number)	Continuous
13–24	STD sub-types (12 binary)	Binary
25	STDs: Number of diagnosis	Continuous
26	Dx:Cancer	Binary
27	Dx:CIN	Binary
28	Dx:HPV	Binary
29	Dx	Binary
30	Hinselmann	Binary (auxiliary target)
31	Schiller	Binary (auxiliary target)
32	Citology	Binary (auxiliary target)
33	**Biopsy**	**Binary (primary target)**

A.2 Saved Model File Reference

Table 9: Model filenames saved to Google Drive

Filename	Description
`scaler.pkl`	StandardScaler (required for all models)
`lr_model.pkl`	Logistic Regression
`svm_model.pkl`	SVM with RBF kernel
`knn_model.pkl`	KNN ($k = 5$)
`nb_model.pkl`	Gaussian Naive Bayes
`dt_model.pkl`	Decision Tree (max_depth=5)
`rf_model.pkl`	Random Forest (200 trees)
`gb_model.pkl`	Gradient Boosting
`xgb_model.pkl`	XGBoost (default params)
`lgbm_model.pkl`	LightGBM
`mlp_model.pkl`	MLP Neural Network
`voting_model.pkl`	Soft Voting Ensemble
`stacking_model.pkl`	Stacking Ensemble
`xgb_tuned_model.pkl`	XGBoost (hyperparameter-tuned)